

约束满足问题的求解

祝恩
计算机学院
国防科技大学



学习内容

1. CSP的回溯搜索
2. 提高回溯搜索效率：
变量顺序、值的顺序、提前检查失败(前向检验、弧相容维护)
3. CSP的局部搜索
4. CSP的问题结构

1 CSP的回溯搜索



形式化为一个搜索问题，对其进行改进

形式化为一个标准搜索

- States are defined by the values assigned so far
- Initial state: the empty assignment, $\{\}$
- Transition model
 - assign a value to an unassigned variable
 - fail if no legal assignment
- Goal test: the current assignment is complete and satisfies all constraints
- Action Cost
- Actions(s)

ACTIONS(s)

- Suppose there are 3 variables, and each has 2 values. How many actions at start?

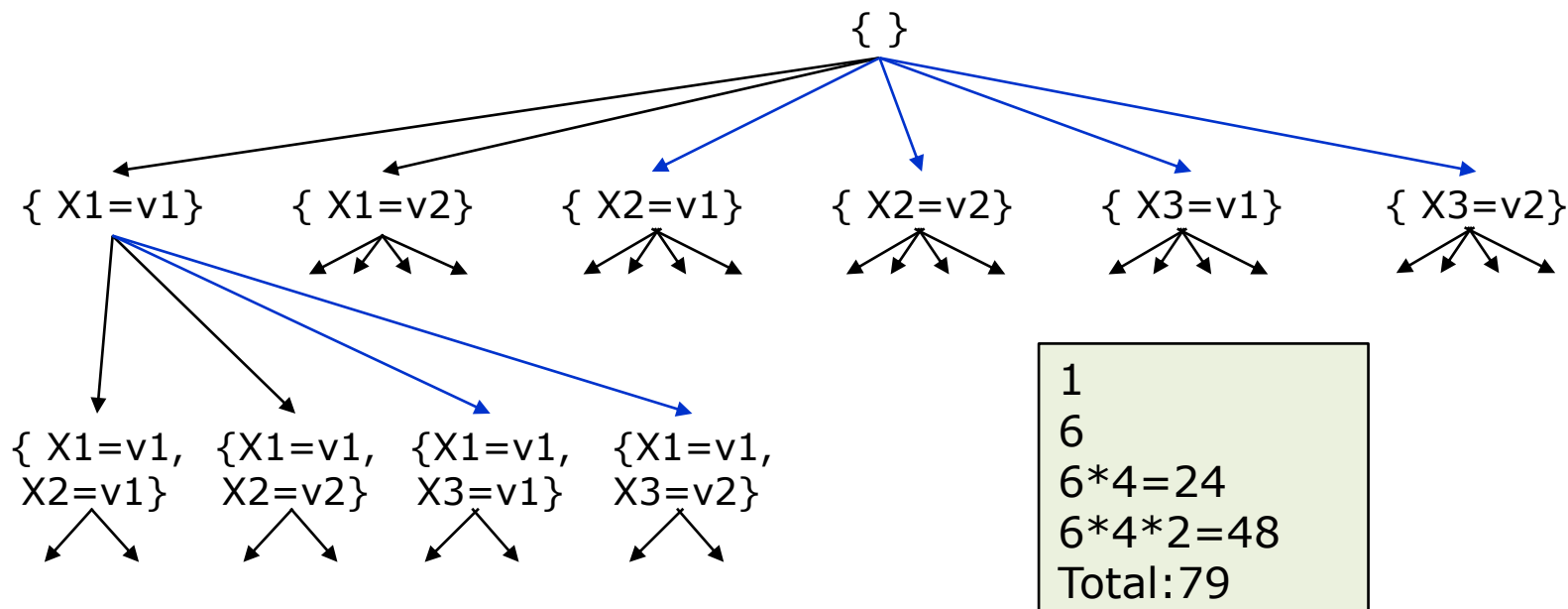
当前状态下有多少个动作？

搜索树有多少个结点？

能否降低分支因子，以降低搜索树规模？

ACTIONS(s)

- Suppose there are 3 variables, and each has 2 values.

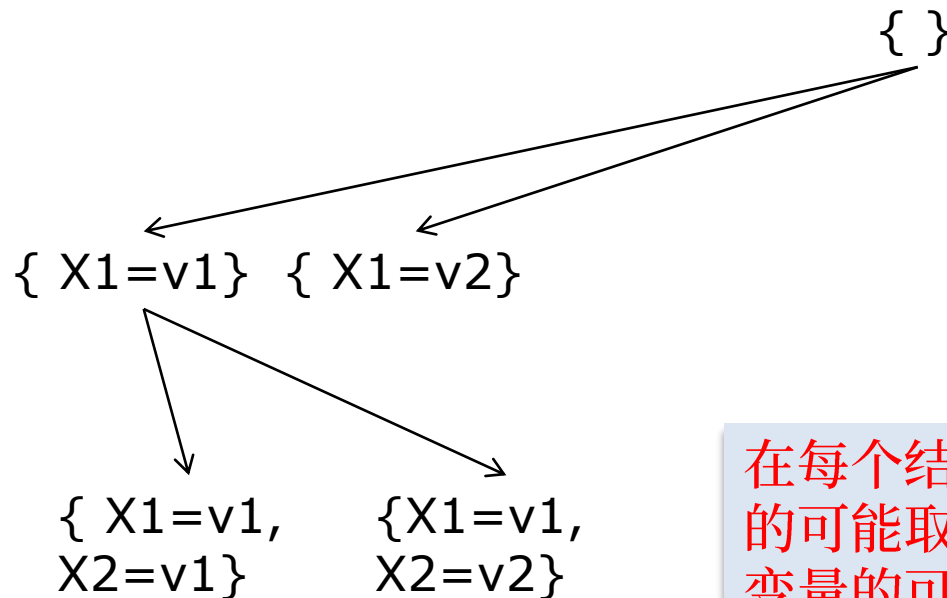


ACTIONS(s)中动作的数量等于未赋值的变量数
*平均每个变量的值域大小。能否降低分支因子，
以降低搜索树的规模？



每步只考虑单个变量

□ suppose there are 3 variables, and each has 2 values.



1
2
4
8
Total: 15

在每个结点，只考虑某个未赋值变量的可能取值，而不是考虑所有未赋值变量的可能取值。

变量赋值的顺序是可以交换的。交换变量赋值的顺序不改变问题的结果。

回溯搜索

Backtracking Search

- Idea 1: Only consider a **single variable** at each point:
 - Variable assignments are **commutative**
[WA = red then NT = green] same as
[NT = green then WA = red]
 - Only need to consider assignments to a single variable at each step
- Idea 2: Only allow **legal assignments** at each point
 - consider only values which do not conflict previous assignments, might have to do some computation
- Depth-first search for CSPs with these two improvements is called **backtracking search**

回溯搜索算法

function BACKTRACKING-SEARCH(csp) **returns** a solution, or failure
 return BACKTRACK({ }, csp)

function BACKTRACK(assignment, csp) **returns** a solution, or failure
 if assignment is complete **then return** assignment
 var $\leftarrow$ **SELECT-UNASSIGNED-VARIABLE**(csp)
 for each value in **ORDER-DOMAIN-VALUES**(var, assignment, csp) **do**
 if value is consistent with assignment **then**
 add {var = value} to assignment
 inferences $\leftarrow$ **INFERENCE**(csp, var, value)
 if inferences $\neq$ failure **then**
 add inferences to assignment
 result $\leftarrow$ BACKTRACK(assignment, csp)
 if result $\neq$ failure **then**
 return result
 remove {var = value} and inferences from assignment
 return failure

函数INFERENCE可以选择性地被设计成弧相容、路径相容或K-相容。如果一个赋值失败了（不管是在INFERENCE还是在BACKTRACK中），则从当前赋值中删除这个赋值（包括从INFERENCE得到的）并尝试新的赋值

2 提高回溯搜索效率



启发式；如何尽早发现解？

如何提高搜索效率？

function BACKTRACKING-SEARCH(csp) **returns** a solution, or failure
 return BACKTRACK({ }, csp)

function BACKTRACK(assignment, csp) **returns** a solution, or failure
 if assignment is complete **then return** assignment
 var $\leftarrow$ **SELECT-UNASSIGNED-VARIABLE**(csp)
 for each value in **ORDER-DOMAIN-VALUES**(var, assignment, csp) **do**
 if value is consistent with assignment **then**
 add {var = value} to assignment
 inferences $\leftarrow$ **INFERENCE**(csp, var, value)
 if inferences $\neq$ failure **then**
 add inferences to assignment
 result $\leftarrow$ BACKTRACK(assignment, csp)
 if result $\neq$ failure **then**
 return result
 remove {var = value} and inferences from assignment
 return failure

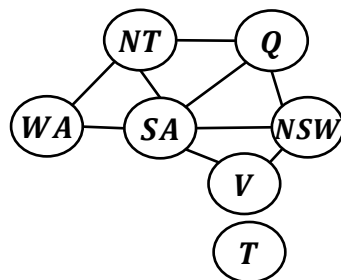
如何提高回溯搜索的效率

- General-purpose ideas can give huge gains in speed:
 - Which **variable** should be assigned next?
 - In what order should its **values** be tried?
 - Can we **detect inevitable failure early**?

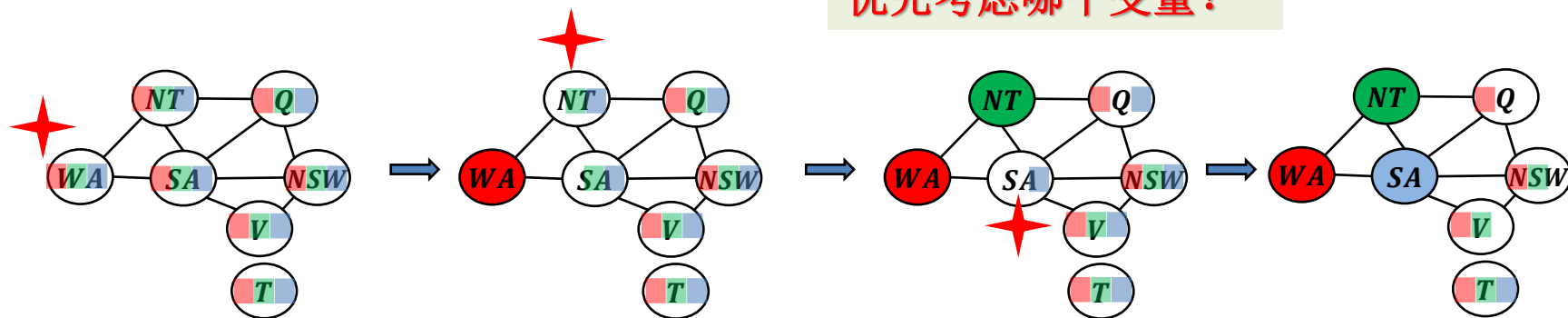


变量的次序

变量的次序



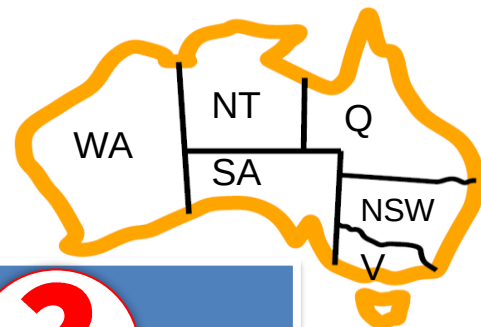
优先考虑哪个变量？



最少剩余值变量优先

变量的次序：最少剩余值变量优先

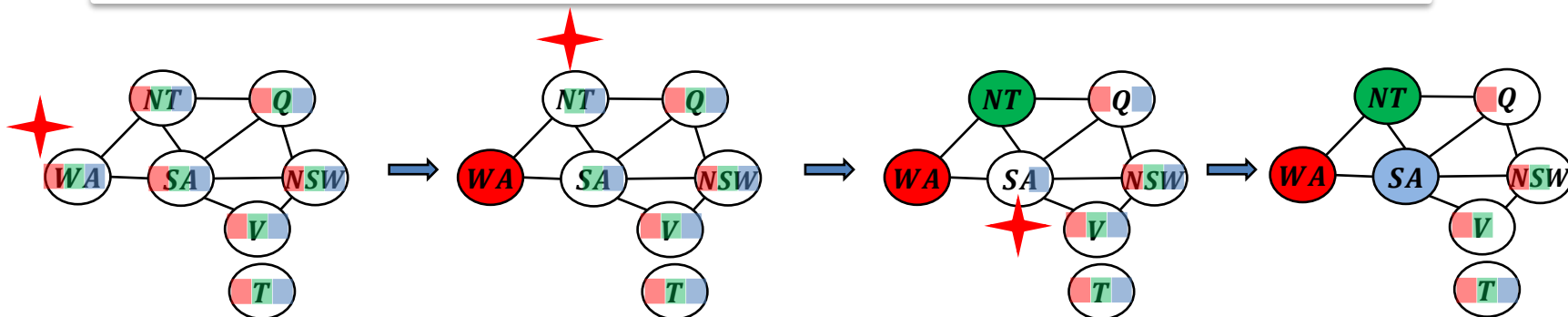
Minimum remaining values



- 最少剩余值(MRV)变量优先：
优先选择剩余值数量最少的变量。

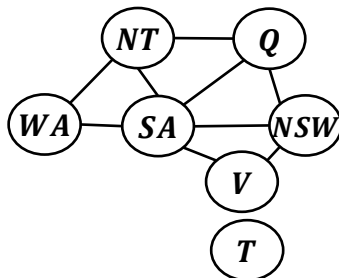


- 选择剩余值数量多的变量容易使得剩余值数量少的变量的值域变为空值域。
- 也称为“最受约束变量”，受约束多的变量被删除的值就多，剩余值就少。
- 或称为“失败优先”启发式，剩余值少的变量的值域容易变为空而失败。

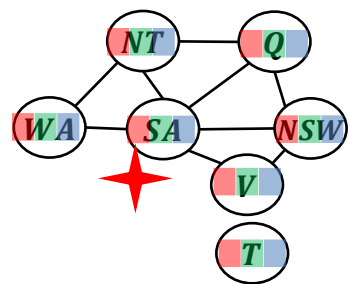


最少剩余值变量优先

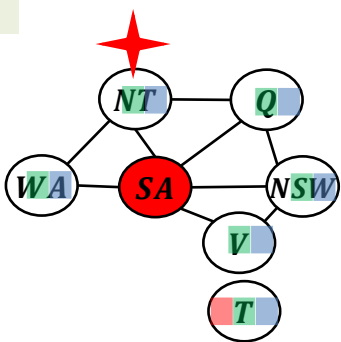
如果剩余值数量相同



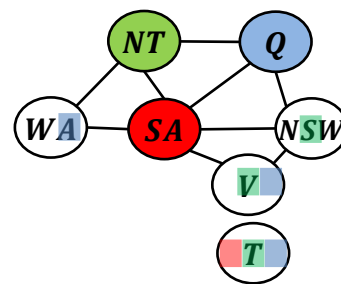
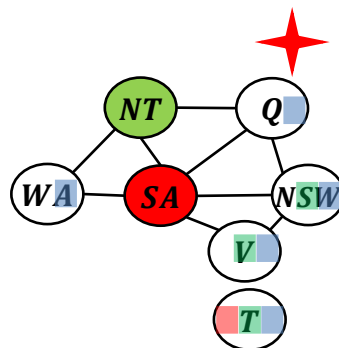
优先考虑哪个变量？



最大度变量优先



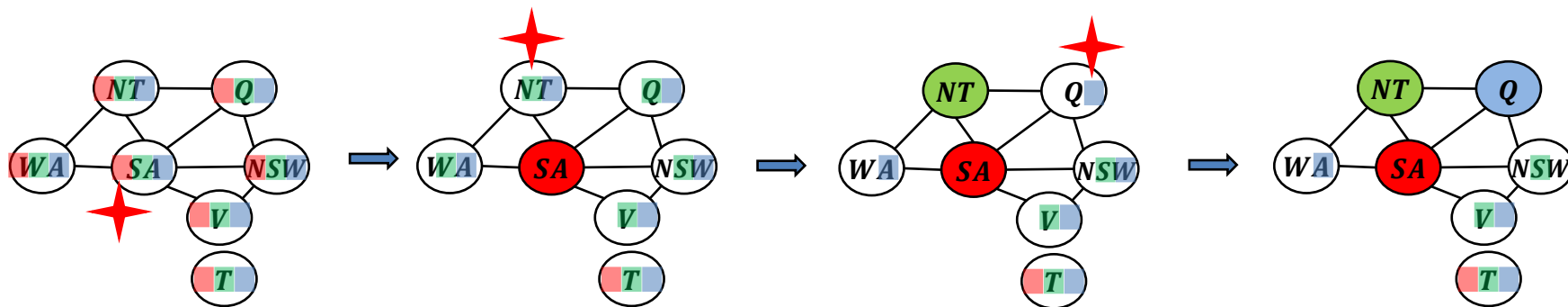
最大度变量优先



如果剩余值数量相同：最大度变量优先

degree heuristic

- 最大度变量优先：优先选择对其他变量约束最多的变量（约束最多其他变量的变量）。
- 通过选择与其它未赋值变量约束最多的变量来**试图降低未来的分支因子**



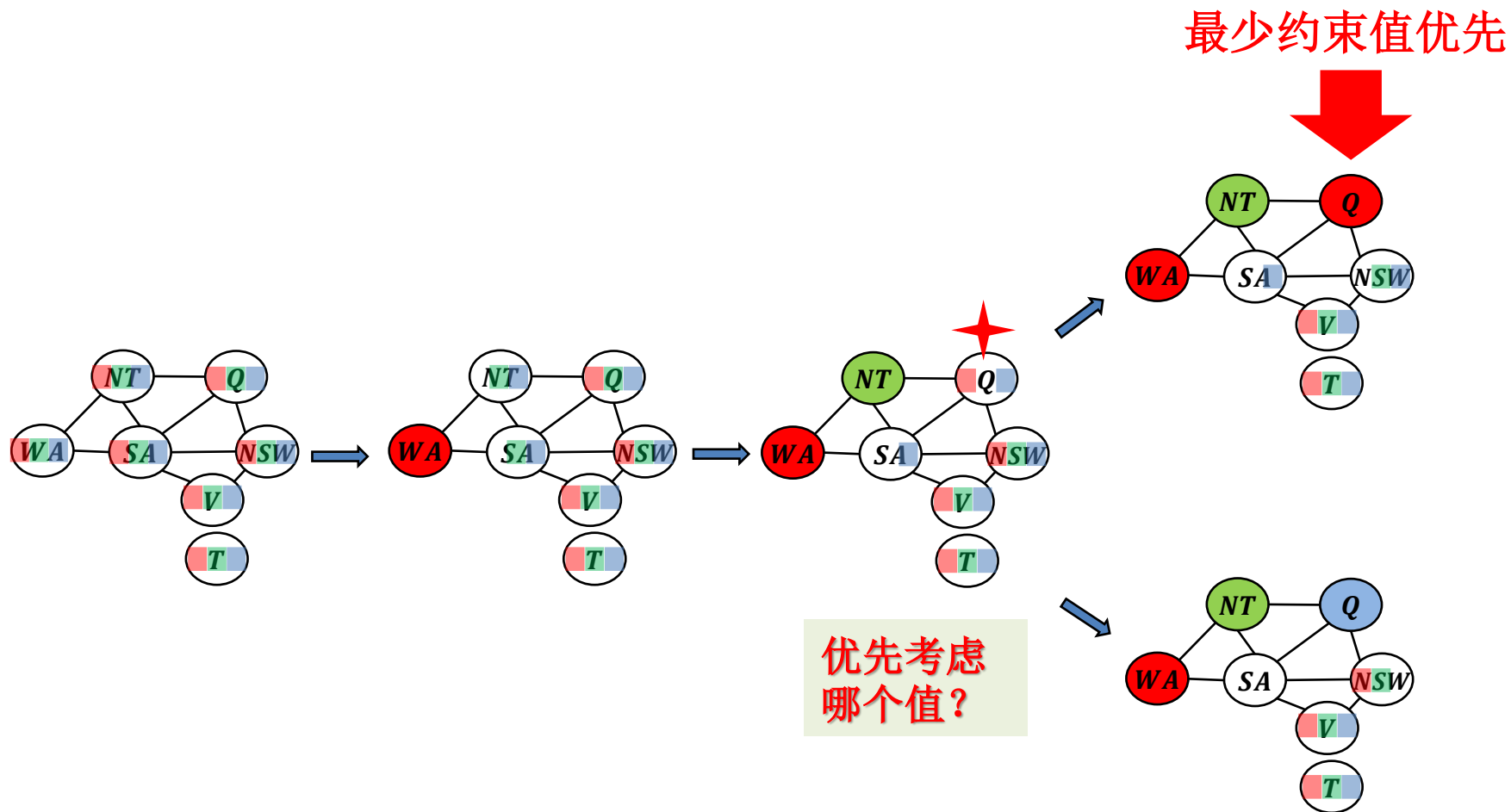
最大度变量优先

最大度变量优先

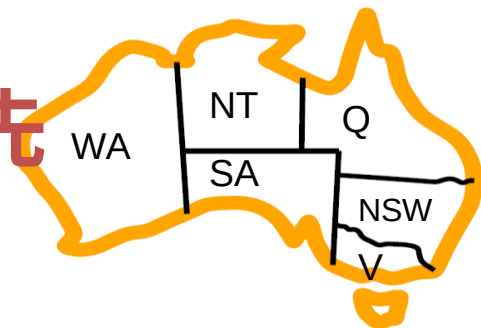


值的次序

值的次序

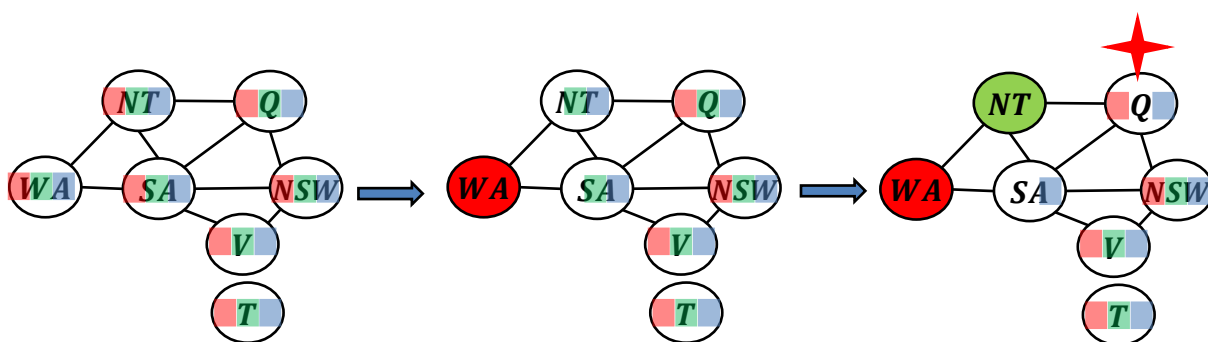


值的次序：最少约束值优先



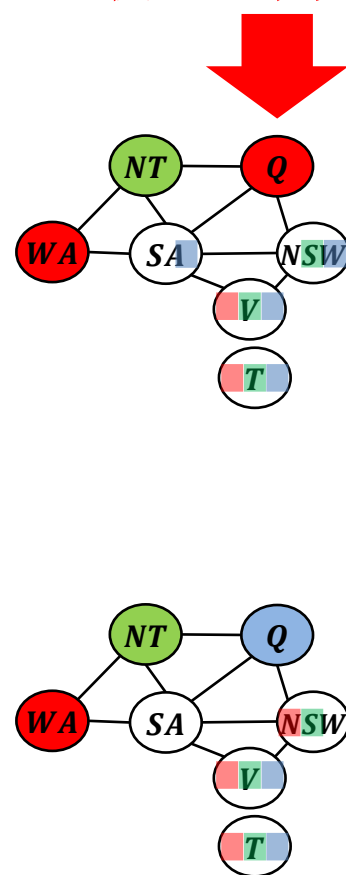
least constraining value

最少约束值优先：选择的取值会删除邻居变量的一些有效值，选择的取值应尽可能少地删除邻居变量的有效值（让它们保留更多的剩余值）。



优先考虑
哪个值？

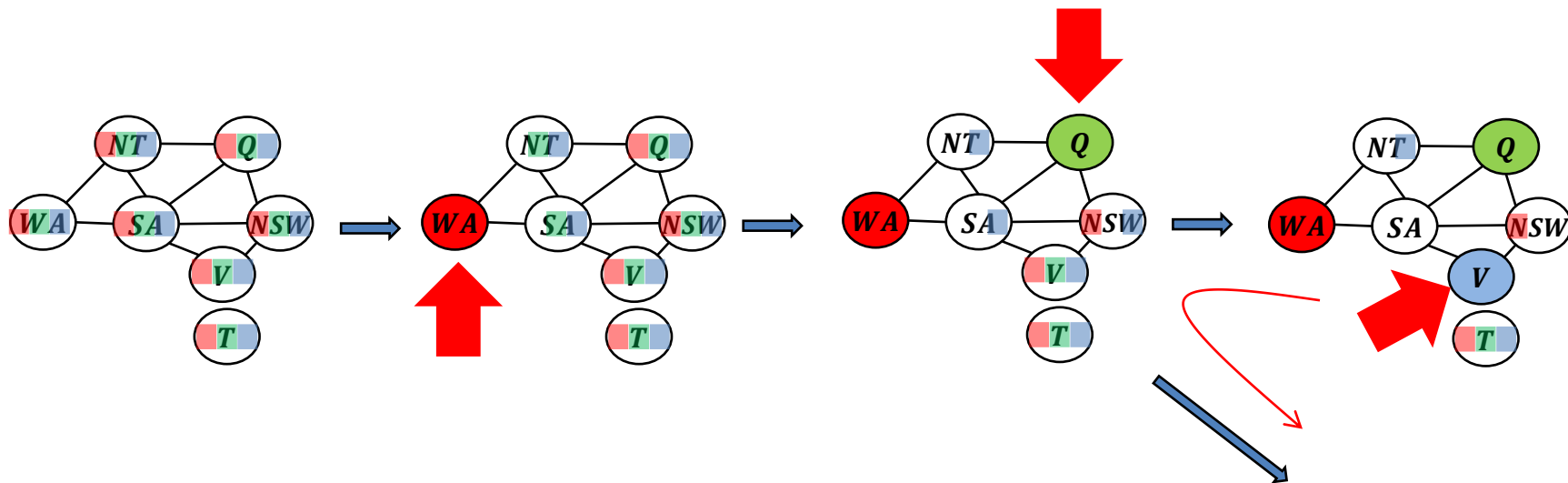
最少约束值优先





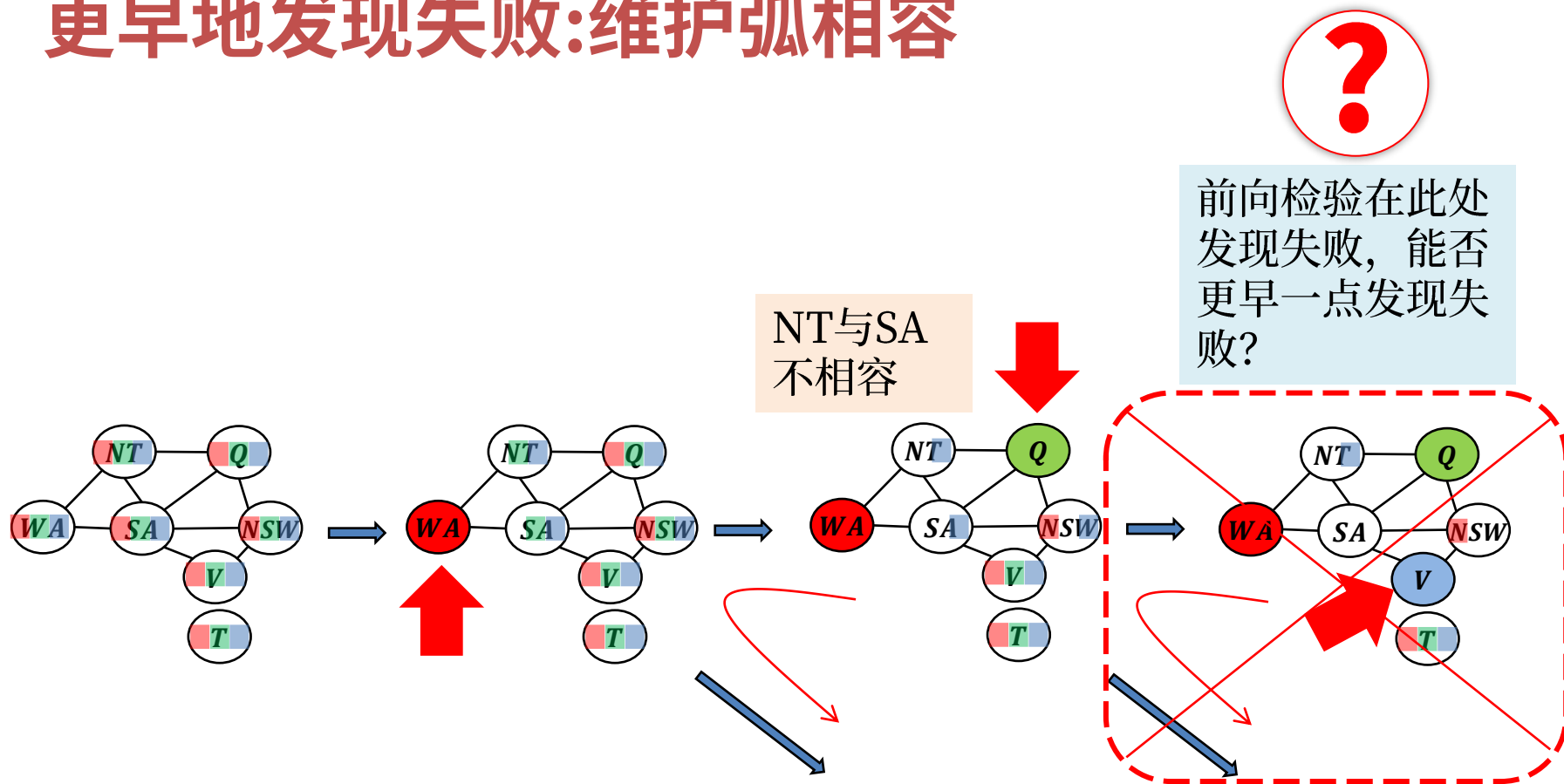
**如果注定失败，尽早回溯
尽早考虑下一个值或回到父结点**

提前发现失败：前向检验



前向检验(Forward Checking): 一个变量X赋值后, 对每个相邻变量Y根据弧相容推理从值域删除与X不相容的值, 如果某个变量值域变为空, 就回溯。

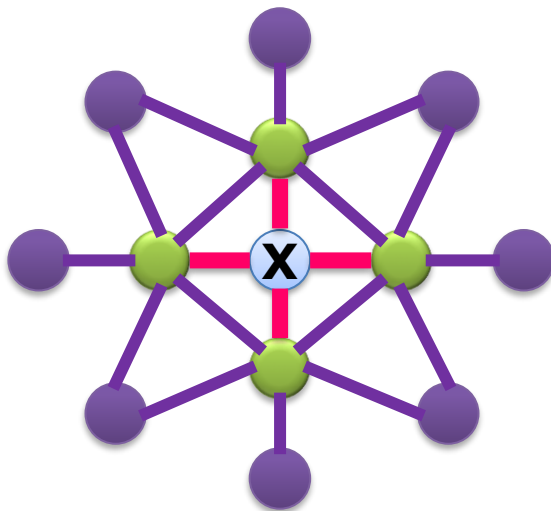
更早地发现失败:维护弧相容



维护弧相容(MAC): 当变量 x 赋值后, INFERENC调用AC-3, 从与 x 相邻的所有未赋值变量开始, 进行约束传播, 一旦某个变量的值域变为空, 则AC-3调用失败并立即回溯

前向检验 vs. 维护弧相容

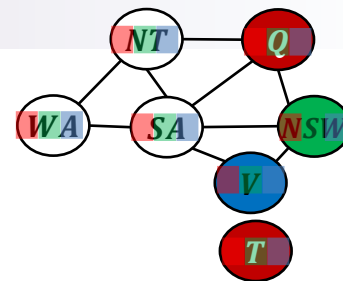
前向检验与MAC的第一步相同，
但前向检验在X的相邻变量值域
发生变化时并不递归传播约束。





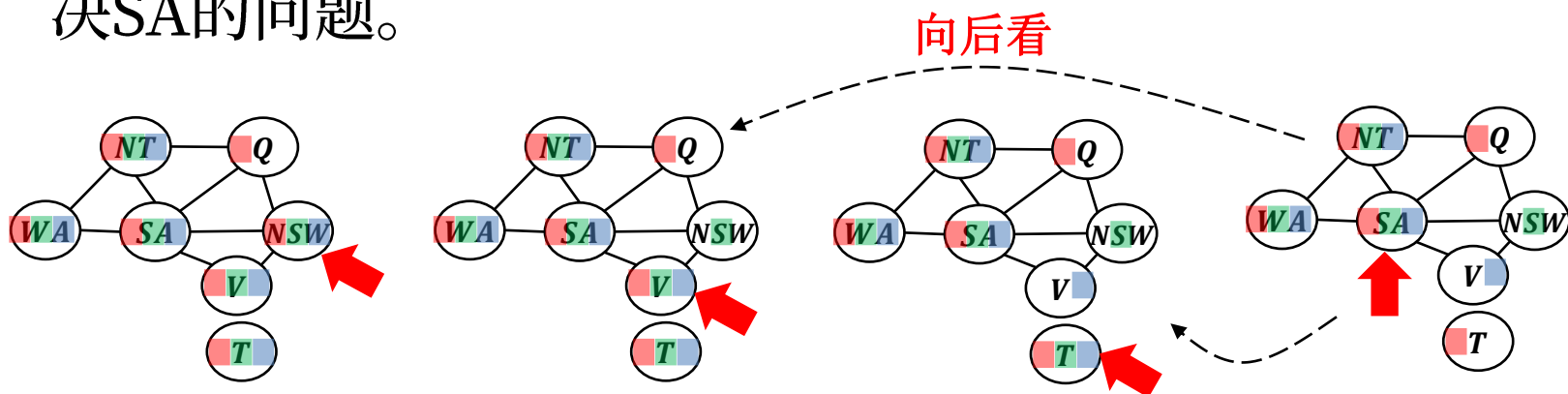
智能回溯：向后看

智能回溯



□ BACKTRACKING-SEARCH 算法当一个分支上的搜索失败时退回前一个变量并且尝试另外一个值。这称为时序回溯，因为重新访问的是时间最近的决策点。

□ 按照固定的变量顺序 *Q*、*NSW*、*V*、*T*、*SA*、*WA*、*NT*，应用简单回溯算法求解。假设已经进行了部分赋值 $\{Q = red, NSW = green, V = blue, T = red\}$ 。当处理下一个变量 *SA* 时，发现任何值都违反约束条件。我们退回到 *T*，试着给 Tasmania 赋一种新的颜色！但对 *T* 重新着色不能解决 *SA* 的问题。



3 CSP的局部搜索



CSP的局部搜索算法;
逃离高原的方法;
问题改变时局部搜索的优势

CSP的局部搜索算法

- 在为变量选择新值的时候，选择能够最少化与其它变量冲突的值——最少化冲突启发式。

function MIN-CONFLICTS(*csp*, *max steps*) **returns** a solution or failure

inputs: *csp*, a constraint satisfaction problem

max_steps, the number of steps allowed before giving up

current $\leftarrow$ an initial complete assignment for *csp*

for *i* = 1 to *max_steps* **do**

if *current* is a solution for *csp* **then return** *current*

var $\leftarrow$ a randomly chosen conflicted variable from *csp*.VARIABLES

value $\leftarrow$ the value *v* for *var* that minimizes CONFLICTS(*var*, *v*, *current*, *csp*)

 set *var* = *value* in *current*

return failure

函数Conflicts统计var=v违反的约束的数量

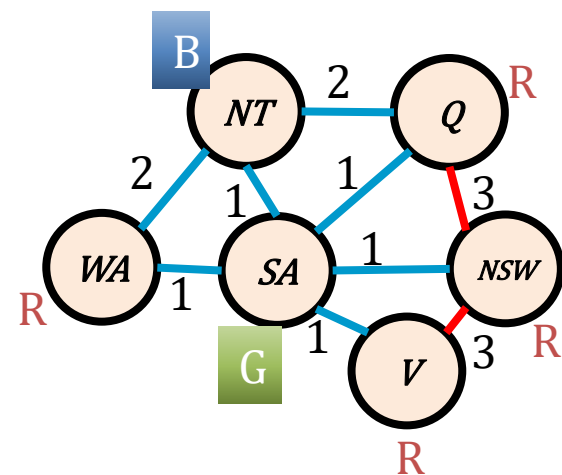
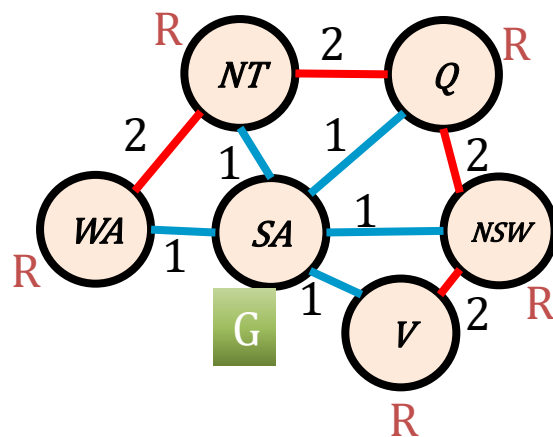
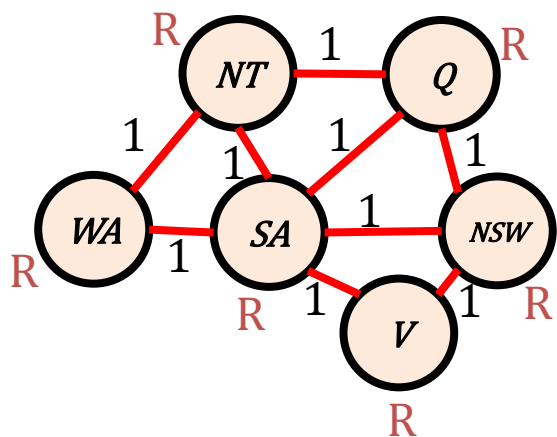
可能有上百万种赋值，它们都只有一个冲突，
——高原问题，如何逃离高原？

如何逃离高原：禁忌搜索和模拟退火

1. 高原搜索——侧向移动到另一个分数相同的状态——可以帮助局部搜索离开高原。这可以由禁忌搜索引导：记录最近访问过的状态，禁止算法再次回到这些状态。
2. 模拟退火也可以用于逃离高原。

如何逃离高原：约束加权

3. **约束加权**，帮助搜索把精力集中在重要约束上。每个约束都有一个权重 W_i ，初始都为1。在搜索的每一步，
- (1) 选择一个变量，修改它的值，使得违反约束的权重和最小化。
 - (2) 接着增加当前赋值违反的约束的权重值。
- 这有两个好处：给高原增加了地形，确保能够从当前状态改善，并且它随着时间的进行不断给难于解决的约束增加权重。



局部搜索 vs. 回溯搜索

- 机场航班日程安排问题：
- 一周的航班日程表可能涉及上千航班和上万人员的安排，但是一个恶劣天气可能就会打乱原来的机场日程安排。如何快速修正日程？
 - a. 局部搜索？
 - b. 回溯搜索？

4 CSP的问题结构



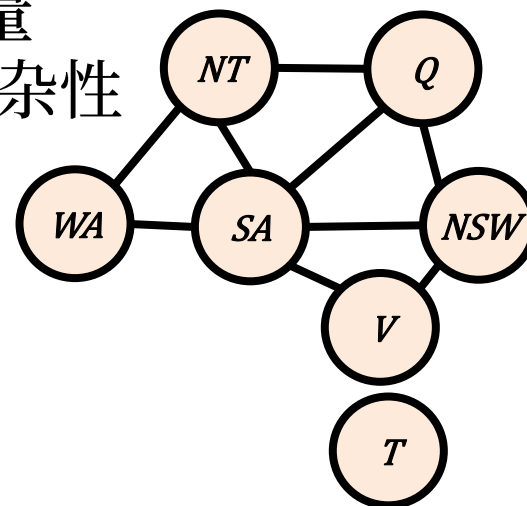
独立子问题

树形结构的CSP的求解算法

化简为树形结构的方法：删除结点(环割集)；合并结点(树分解)

独立的子问题

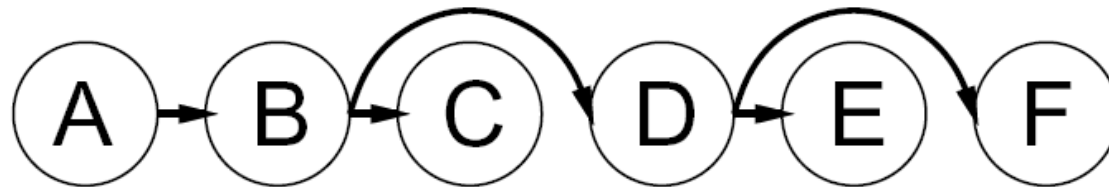
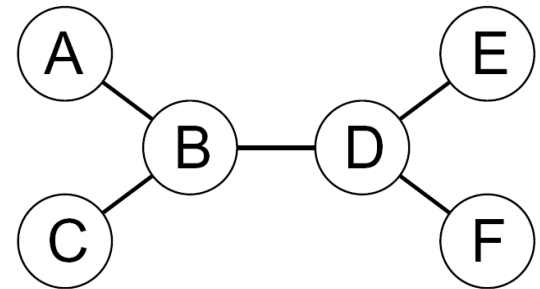
- 独立的子问题 $CSP_1, CSP_2, \dots$
- 独立性通过寻找连通子图确定
- 如果赋值 S_i 是 CSP_i 的一个解，那么 $\cup_i S_i$ 是 $\cup_i CSP_i$ 的一个解。
- 假设变量共 n 个，每个子问题 c 个变量
一共 n/c 个子问题，每个子问题的复杂性为 $O(b^c)$ ，总工作量为 $O(b^c n/c)$
- 不分解的情况下为 $O(b^n)$
- 实例： $n=80, c=20, b=2$
 - $O(b^c n/c)$: $2^{20} * 4$
 - $O(b^n)$: 2^{80}



两个独立的子问题

树形结构的CSP

1. Choose a variable as root, order variables from root to leaves such that every node's parent precedes it in the ordering

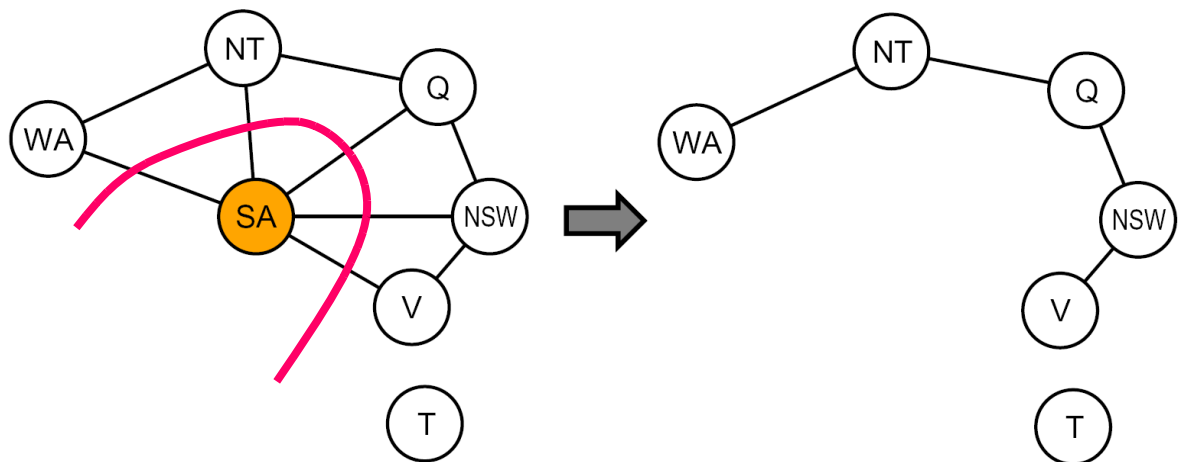


2. For $i = n : 2$, Make-Arc-Consistent(Parent(X_i), X_i)
 3. For $i = 1 : n$, assign X_i consistently with Parent(X_i)
- Runtime: $O(n b^2)$

Make_Arc_Consistent(X,Y):
通过最少地删除X的值域中的值,
使得X关于Y是弧相容的。



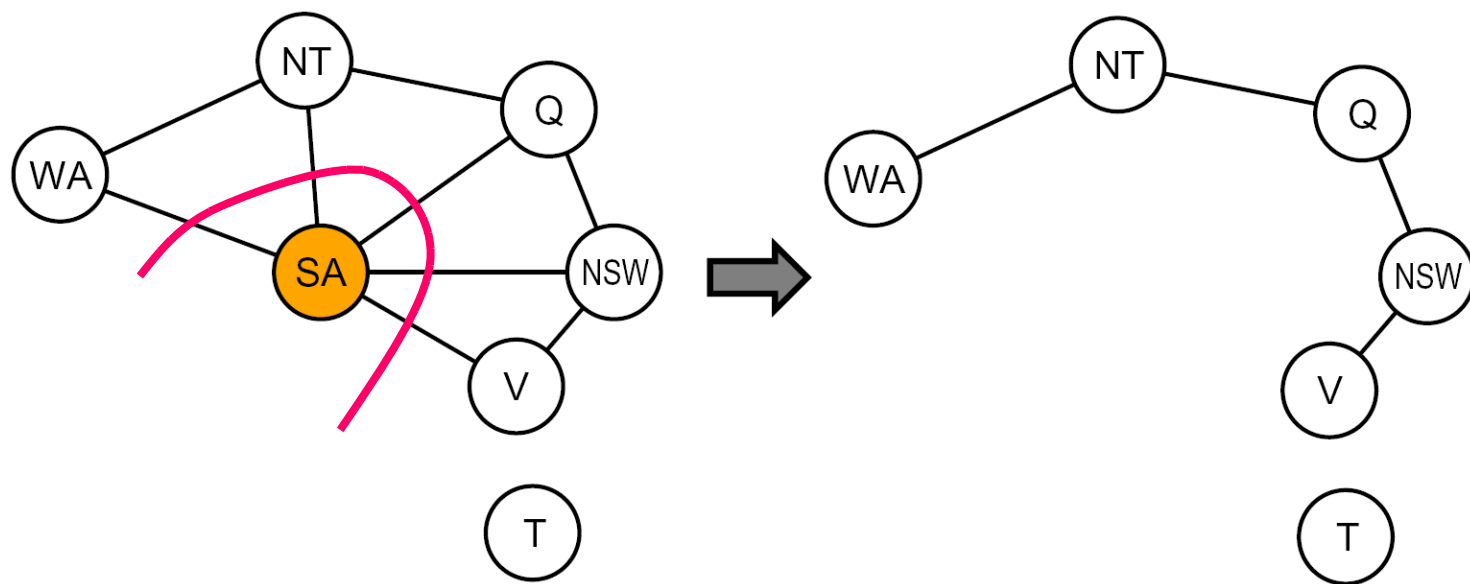
如果不是树形结构的约束图
如何化简为树的形式？



基于删除结点的方法：环割集

基于删除结点的方法

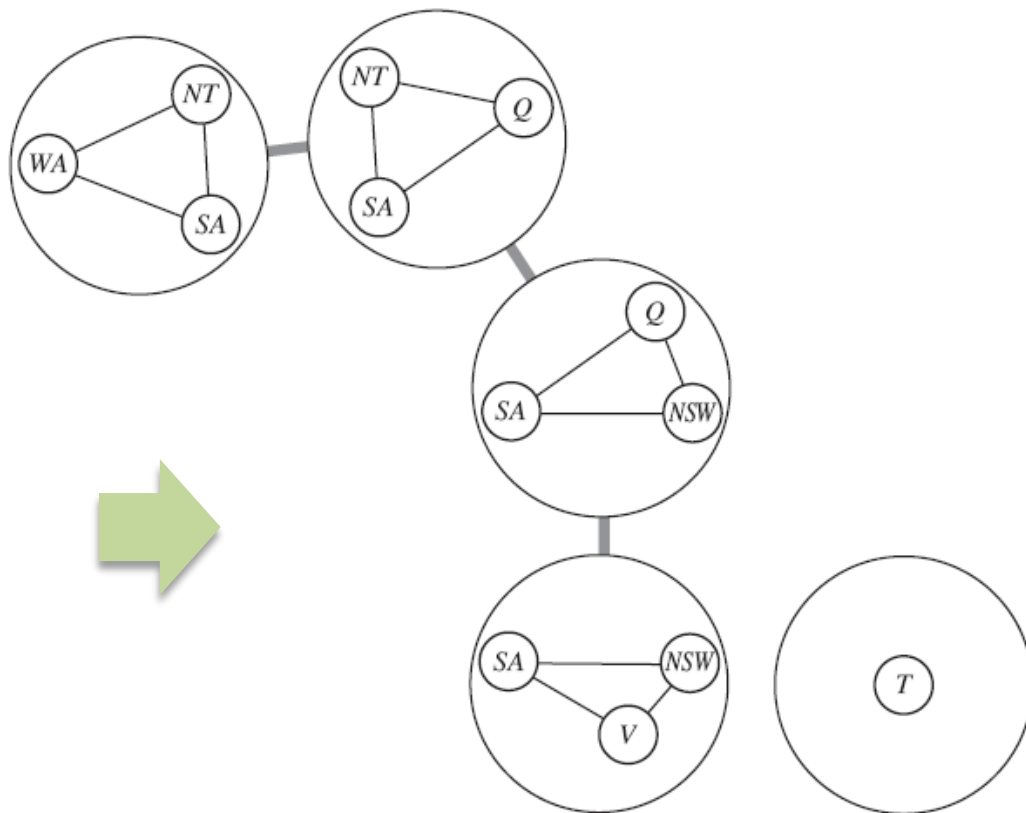
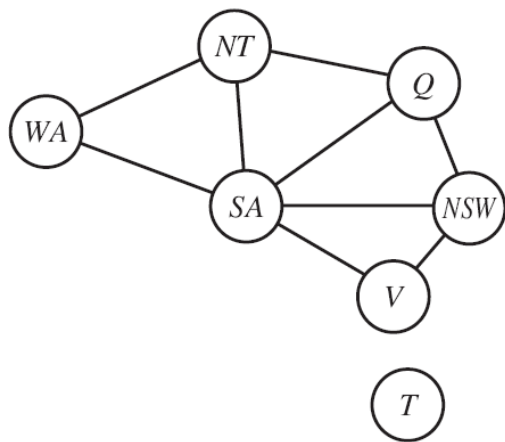
- 先对**部分变量**赋值，使剩下的变量能形成一颗树
 - 例如给SA赋一个固定的值，修剪其他变量的值域
 - 其他变量的CSP的每个解(用树形结构的算法求解)都与SA的值相容
 - 其他变量的CSP可能无解(此实例中是有解的)，这时需要尝试SA的其他值



一般化算法

1. 从CSP的变量中选择子集 S ，使得约束图在删除 S 之后成为一棵树。 S 称为环割集（**cycle cutset**）。
2. 对于 S 中变量的满足约束的每种可能赋值，
 - ① 从CSP剩余变量的值域中删除与 S 的赋值不相容的值；
 - ② 如果剩余CSP有解，把解和 S 的赋值一起返回。

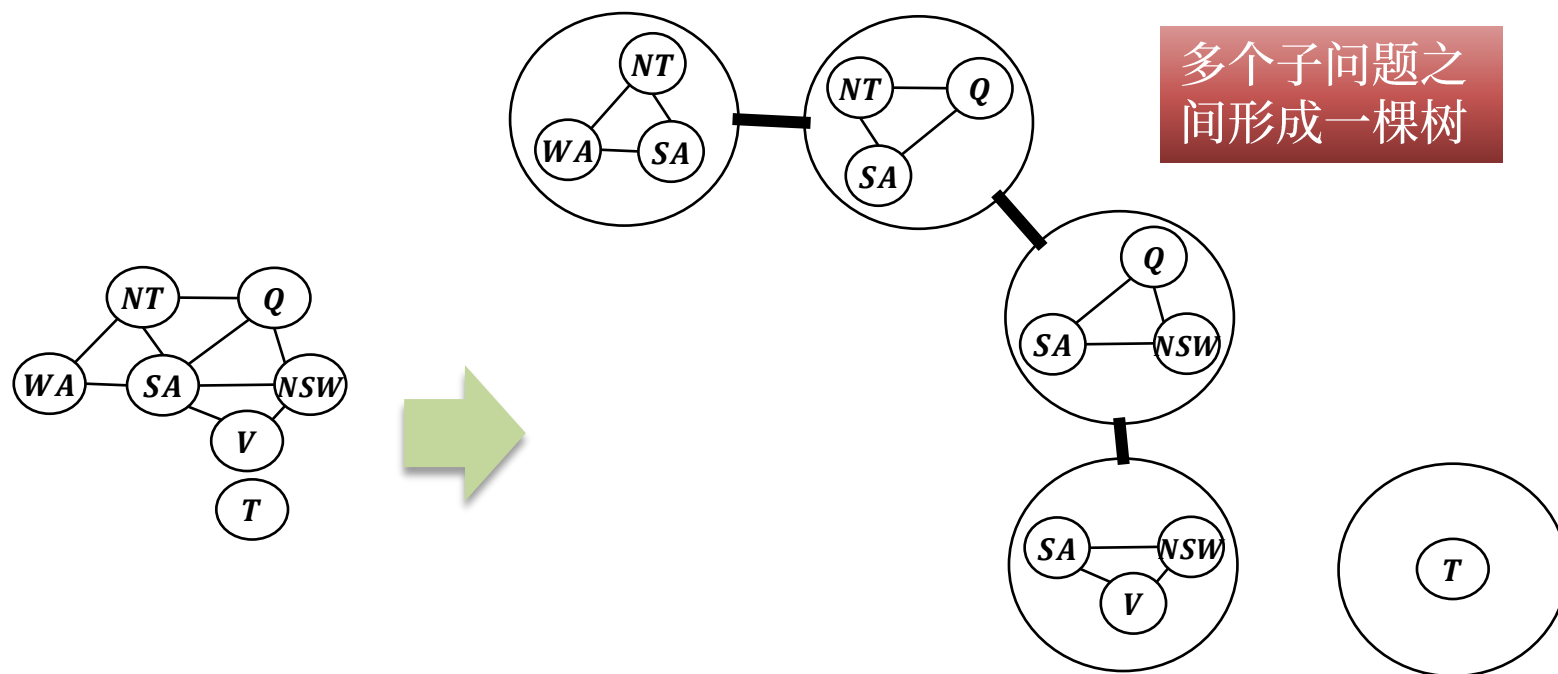
- 假设 n 个变量，每个变量 b 个值。如果环割集的大小为 c ，那么总的运行时间为： $O(b^c \cdot (n-c)b^2)$
- 约束图近似为一棵树时 c 很小。另一个极端是 c 很大，最大是 $n-2$ 。找到 c 最小的环割集是NP-hard的，有一些高效的近似算法。



基于合并结点的方法：树分解

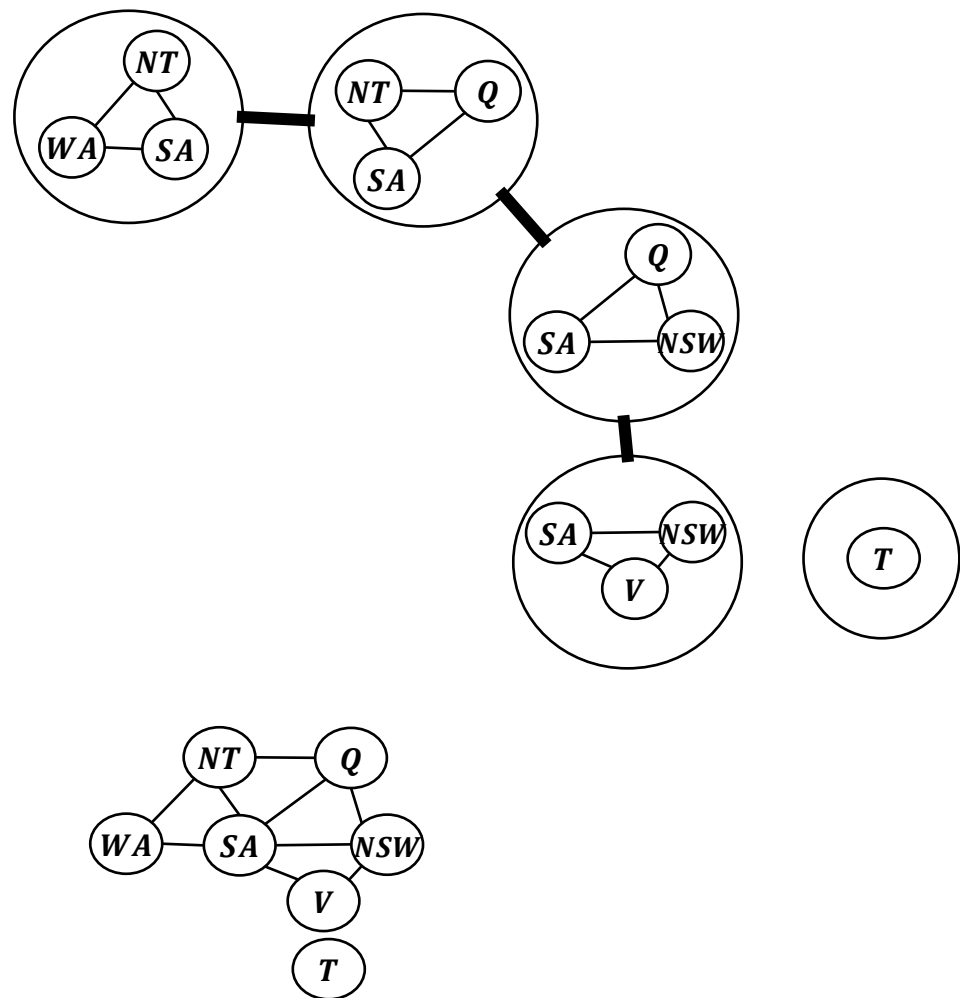
树分解

- 树分解将约束图分解为**相关联**的子问题，每个子问题独立求解，再把得到的结果合并起来。
 - 例如，地图着色问题分解为5个相关联的子问题



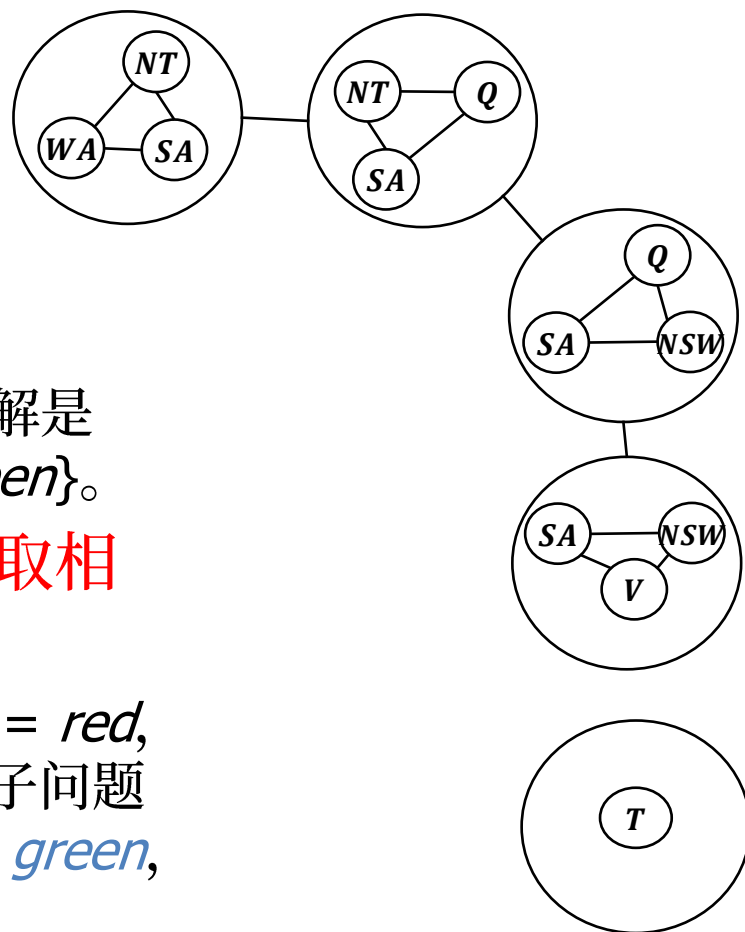
树分解必须满足的条件

1. 原问题中的**每个变量**至少在一个子问题中出现
2. 原问题中的**每个约束**至少在一个子问题中出现。
3. **如果一个变量出现在两个子问题中**，那么连接这两个子问题的**路径上的所有子问题**里都必须出现这个变量。(反应了这个约束：在该路径上每个子问题中出现的同一个变量必须有相同的值。)

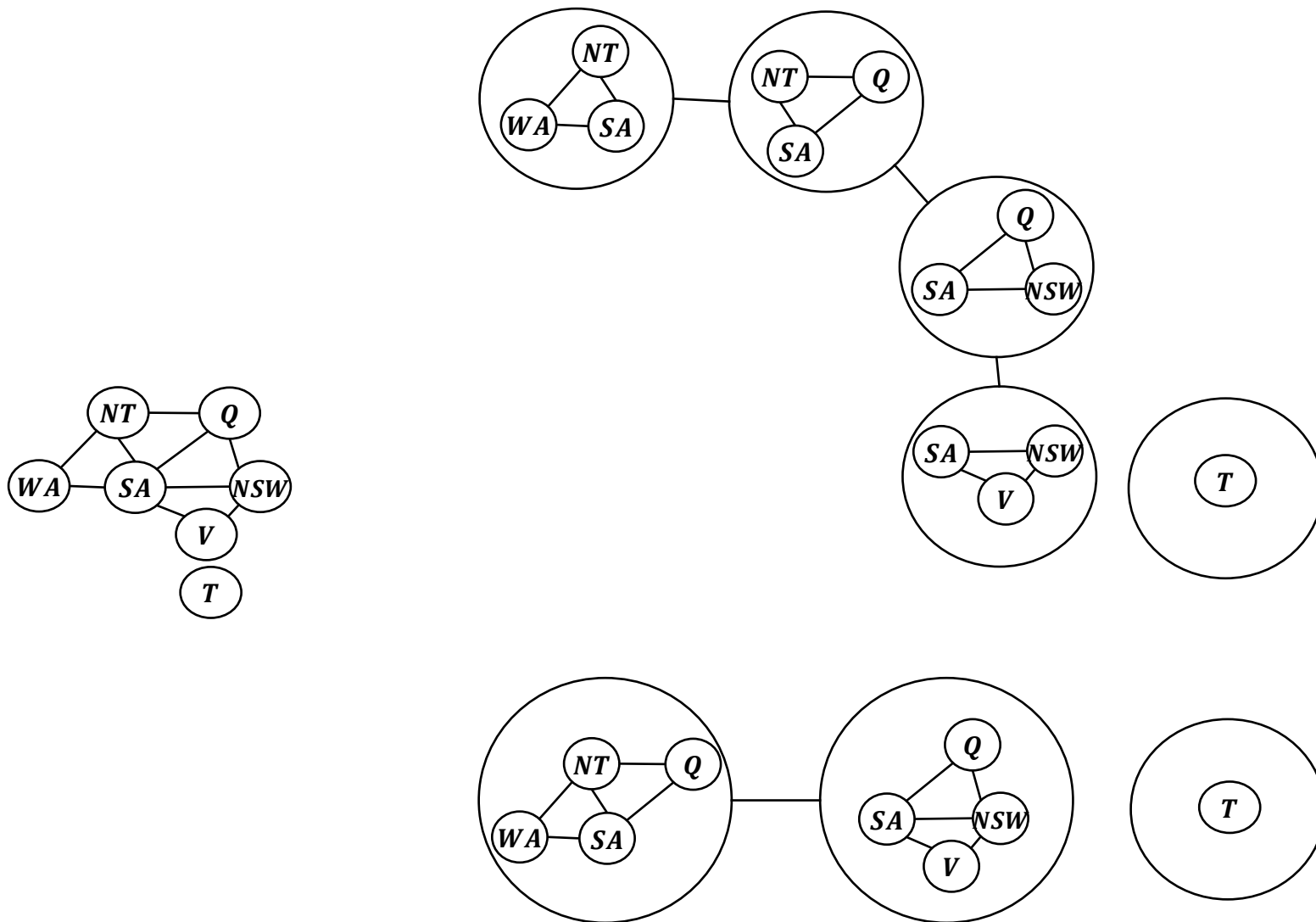


树分解CSP的求解

- **变量**: 每个子问题被视为一个“巨型变量”，
- **值域**: 这个巨型变量的值域是这个子问题的**所有解的集合**。
 - 例如，图中最左边的子问题的一个解是 $\{WA = red, SA = blue, NT = green\}$ 。
- **约束**: 这些子问题的**共享变量要取相同的值**。
 - 例如，给定第一个子问题的解 $\{WA = red, SA = blue, NT = green\}$ ，下一个子问题的相容解只能是 $\{SA = blue, NT = green, Q = red\}$ 。
- **求解**: 用前面给出的**树算法**求解。



多种可能的树分解



Thanks
I U 9 U K 2